

Подготовка к защите игры Общага

Руководство для начинающего программиста

byVanoGame | Python 3.12 | Pygame 2.6.1 | Версия от 9 сентября 2026 года

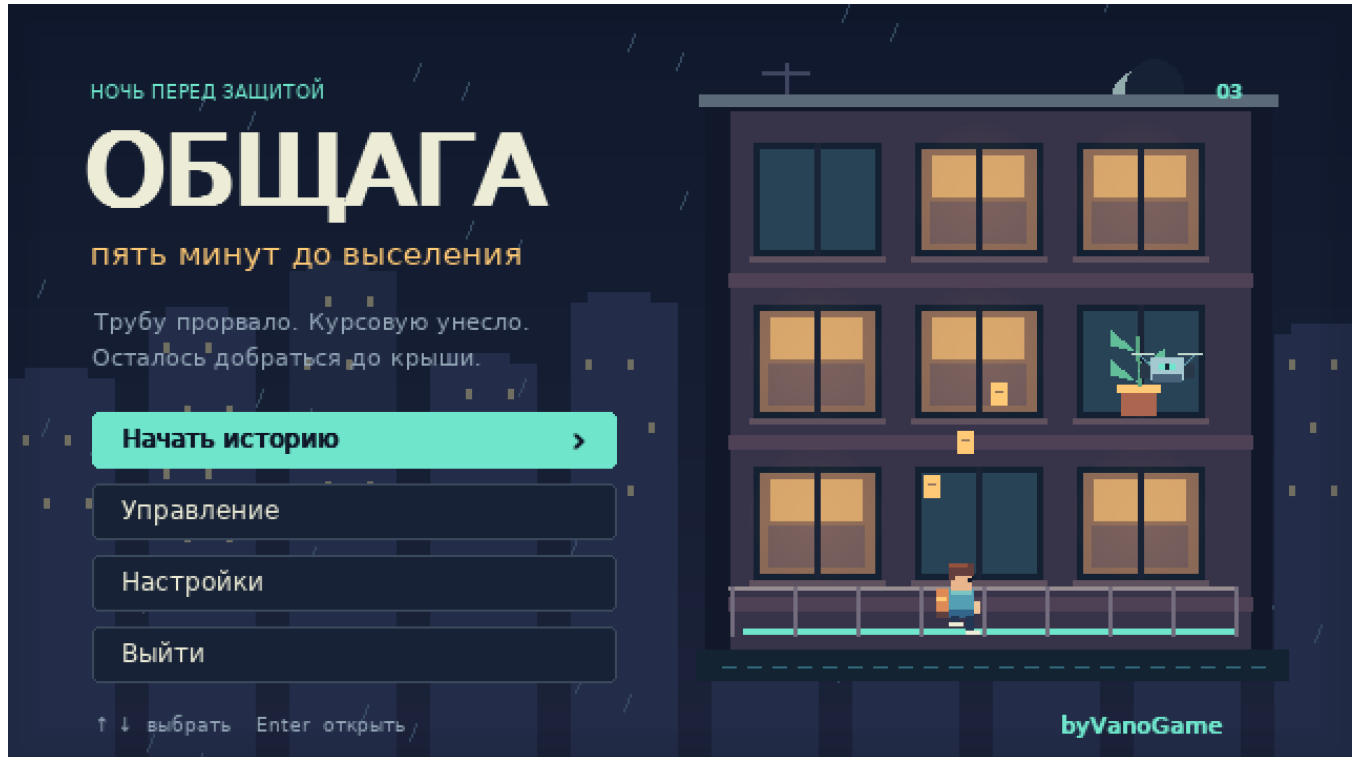


Рисунок 1. Главное меню актуальной версии с подписью byVanoGame.

Это руководство помогает подготовиться к защите игры «Общага: пять минут до выселения». Здесь объясняется, как запустить проект, что показать преподавателю и как читать его код. Предполагается, что читатель почти не программировал.

Для первого знакомства прочитайте основные главы по порядку. Справочник в конце содержит каждый явно объявленный метод и функцию игрового пакета: назначение, сигнатуру, файл и номера строк. Короткая речь и быстрые ответы вынесены во второй Word-файл.

1 Как подготовиться за один вечер

Цель подготовки — уметь своими словами связать действие в игре с конкретным местом в коде. Например: «Нажал пробел → появилась команда → физика задала скорость вверх → Pygame нарисовал новую позицию».

Время	Что сделать	Что должно получиться
10 минут	Запустить игру и прочитать управление	Двигаться, прыгать, открыть паузу и вернуться
15 минут	Прочитать главы о структуре, времени и физике	Объяснить модель, координаты, шаг и столкновение
15 минут	Разобрать Enemy и пример A*	Назвать наследников и объяснить g, h и f
10 минут	Найти в коде пять мест из сценария показа	Открыть нужный файл без долгого поиска
10 минут	Произнести короткую речь и ответить без подсказки	Уложиться примерно в две минуты

Что выучить обязательно

Где лежат правила; почему Pygame отделён от модели; зачем нужен фиксированный шаг; как работают прямоугольные коллизии; где наследование; как A* выбирает следующую клетку. Не требуется запоминать все координаты карт и команды рисования растений.

Как искать информацию в Word

Нажмите Ctrl+F и ищите имя метода, например `update_player` или `astar`. В области навигации Word можно переключиться на заголовки. Номера слева от кода относятся к строкам исходного Python-файла, а не к страницам руководства.

Речь — опора для выступления. Перед защитой обязательно повторите действия в игре и объяснения своими словами: заученный текст сам по себе не помогает отвечать на уточнения.

2 Запуск и файлы для сдачи

В этой рабочей папке уже есть подготовленное окружение. Самый простой запуск — дважды щёлкнуть start.bat. Альтернатива: открыть PowerShell в папке проекта и выполнить команду.

```
.\.venv\Scripts\python.exe main.py
```

Для переноса нужны main.py, start.bat, requirements.txt, папки dormgame, data, assets, tests и tools, а также документация. Папку .venv на другой компьютер переносить не нужно: она связана с местным Python.

Подготовка другого компьютера

Установите Python 3.12. Затем из папки проекта создайте окружение и установите единственную игровую зависимость. Интернет нужен на этапе установки Pygame, во время игры он не нужен.

```
py -3.12 -m venv .venv
```

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

```
.\.venv\Scripts\python.exe main.py
```

Если не запускается

Симптом	Что проверить
Открывается Microsoft Store	Используйте .venv\Scripts\python.exe или установленный py -3.12, а не ярлык python.
No module named pygame	Установите requirements.txt именно интерпретатором из .venv.
No module named dormgame	Откройте терминал в корне проекта. Проверочные сценарии запускайте через -m tools...
Нет звука	Проверьте М и настройки. Без аудиоустройства игра всё равно должна работать.

Скрипт build_defense_docs нужен только для повторной сборки Word. Для запуска игры устанавливать python-docx или офисные библиотеки не требуется.

3 Как играть и что изменилось



Рисунок 2. Первый этаж. Наверху статистика, снизу подсказки, слева необязательный бонус.

Клавиша	Действие
A D или стрелки	Бежать
Пробел	Прыгать; отпускание раньше даёт более низкий прыжок
E	Читать записку и открыть выход
R	Вернуться к контрольной точке
Esc	Пауза
F3	Диагностика физики и A*
M и F11	Звук и полный экран

Выход требует все страницы текущего этажа. На трёх уровнях их 4, 4 и 5. Дополнительно на каждом старте можно пройти немного влево: там билет, кружка или флешка. Сувениры не закрывают выход и дают дополнительную строку в финале.

4 Минимум Python для чтения проекта

Запись	Простой смысл
<code>x = 10</code>	Сохранить число 10 под именем x. Знак <code>=</code> присваивает, а <code>==</code> сравнивает.
<code>if условие</code>	Выполнить вложенные строки, только если условие истинно.
<code>for item in items</code>	По очереди обработать предметы списка.
<code>def</code> и <code>return</code>	Объявить функцию и вернуть её результат вызывающему коду.
<code>class</code> и <code>self</code>	Описать тип объекта; <code>self</code> означает конкретный экземпляр.
<code>True False None</code>	Да, нет и отсутствие значения.
<code>list dict set tuple</code>	Список, словарь ключ → значение, множество без повторов и кортеж.
<code>float int bool str</code>	Дробное число, целое число, логическое значение и строка текста.
<code>_имя</code>	Соглашение о внутреннем методе. Это не запрет доступа на уровне языка.

Как читать реальный короткий метод

dormgame/geometry.py — строки 17–19

```
17     @property
18     def right(self) -> float:
19         return self.x + self.w
```

`def` начинает описание метода. `self` — этот прямоугольник. `self.x` — его левая координата, `self.w` — ширина. `return` возвращает сумму. Подсказка `-> float` сообщает ожидаемый тип результата, но сама ничего не вычисляет.

Над методом в исходнике стоит `@property`: это позволяет писать `box.right`, а не `box.right()`. Отступы в Python задают вложенность. Удаление отступа может изменить смысл программы.

5 Объекты и данные без сложных слов

Класс можно понимать как описание одинаково устроенных объектов. Объект — конкретный экземпляр: один герой, один охранник, одна страница. У каждого свои поля. Метод — действие, связанное с объектом.

dormgame/commands.py — строки 5–11

```
5 @dataclass(frozen=True)
6 class Commands:
7     move: int = 0
8     jump_pressed: bool = False
9     jump_held: bool = False
10    interact: bool = False
11    reset: bool = False
```

Commands — снимок ввода на один шаг. move равен −1, 0 или 1. jump_pressed обозначает новое нажатие, jump_held — текущее удержание. Это разные данные: удержание влияет на высоту, а новый прыжок разрешается только по новому нажатию.

dormgame/model.py — строки 16–21

```
16 class GameEvent:
17     kind: str
18     x: float
19     y: float
20     text: str = ''
21
```

GameEvent сообщает о результате шага: например, page, died или souvenir. Представление получает координаты события и показывает эффект. @dataclass автоматически создаёт обычный конструктор данных; frozen=True запрещает переустановку полей экземпляра.

Класс данных	Что хранит
AABB	х, у, ширину w и высоту h
Body и PlayerBody	Прямоугольник, скорости, опоры; у героя ещё таймеры прыжка
Item	ID предмета, прямоугольник, текст и флаги collected и active
CollisionResult	Были ли контакты по X и Y, приземление и удар о потолок
SearchResult	Путь и число раскрытых узлов
Particle	Положение, скорость, оставшуюся жизнь, цвет и размер частицы

6 Карта проекта и разделение ответственности

Файл	За что отвечает
main.py и app.py	Запуск окна, события устройств, меню и смена экранов
model.py	Состояние сеанса, предметы, смерти, контрольные точки и выход
physics.py	Движение, опора и столкновения
geometry.py	Дробные прямоугольники и пересечения
timing.py и commands.py	Фиксированный шаг, буфер ввода и команды
enemies.py	Охранник, дрон и общий Енему
pathfinding.py	Навигационная сетка и A*
presentation.py и audio.py	Рисование, камера, эффекты и звук
levels.py и data	Чтение, проверка и ручные описания карт
config.py и tests	Параметры и автоматические проверки

Путь одного нажатия

Клавиатура → Application.handle_event → InputBuffer → Commands → GameSession.update → update_player → новые координаты → Renderer.draw_game.

В обратную сторону модель отдаёт GameEvent. Например, collected становится True, модель выдаёт page, затем представление добавляет частицы, а Audio играет звук.

Разделение полезно при изменениях: можно перерисовать студента, не меняя прыжок, или проверить A* без окна. В чистых модулях нет import rpygame. Все изменяемые игровые данные принадлежат объектам сеанса, а не общим глобальным переменным.

7 Как уровень получается из JSON

JSON — текстовый формат данных. В нём нет поведения врага или формулы прыжка. Он отвечает на вопросы: где стоит платформа, где лежит страница и какой тип врага создать.

data/level1.json — начало реального описания уровня

```
1 {
2   "name": "Первый этаж. Всё под контролем",
3   "subtitle": "Вода поднялась. А стипендия — нет.",
4   "width": 1088, "height": 480, "start": [48, 390],
5   "solids": [[0,416,1088,64], [208,384,144,16], [384,352,144,16], [560,320,144,16],
6     [736,288,144,16], [912,256,176,16]],
7   "pages": [
8     {"id":"p1", "rect":[128,394,12,16], "text":"Титульник спасён. Теперь у паники есть
9     официальное название."},
10    {"id":"p2", "rect":[276,362,12,16], "text":"Введение найдено. Хотелось бы сразу
11    заключение."},
12    {"id":"p3", "rect":[620,298,12,16], "text":"Код намок, но отступы остались. Уже
13    неплохо."},
14    {"id":"p4", "rect":[804,266,12,16], "text":"Первый этаж собран. Главное — не потерять
15    нумерацию."}
16  ],
17  "souvenirs": [{"id":"badge1", "kind":"badge", "rect":[8,395,16,18], "text":"Студенческий
18  найден. Теперь охранник знает, кого догоняет."}],
19  "checkpoints": [{"id":"cp1", "rect":[424,326,18,26], "spawn":[420,326]}],
20  "notes": [{"id":"note1", "rect":[84,392,14,24], "text":"Сосед: «Если тонем — без долгов
21  за душ. Студенческий ищи слева от старта."}, {"id":"note1b", "rect":[494,328,14,24],
22  "text":"Дежурный: «Чайник у нас надёжнее сервера. Здесь прогресс не пропадёт.»}],
23  "hazards": [{"id":"water1", "rect":[578,407,100,9]}],
24  "rect = [x, y, w, h]. Например, [128, 394, 12, 16] означает левый верхний угол в точке 128, 394,
25  ширину 12 и высоту 16. ID p1 отличает страницу от других предметов.
```

solids — твёрдая геометрия; **pages** — обязательные страницы; **souvenirs** — необязательный набор; **checkpoints** — сохранения; **notes** — записки; **hazards** — опасности; **exit** — выход.

dormgame/levels.py — строки 124–128

```
124 def load_levels(directory: Path = DATA_DIR) -> list[dict[str, Any]]:
125     levels = [json.loads((directory / f'level{i}.json').read_text(encoding='utf-8')) for i in
126       range(1, 4)]
127     for level in levels:
128       validate_level(level)
129     return levels
```

Валидатор отбрасывает неверные типы, повторные ID, отрицательные размеры, координаты за картой, появление в стене и движение платформы сквозь стену. Наличие данных ещё не доказывает проходимость: для неё есть отдельный сценарий с настоящей физикой.

8 Кадры и фиксированные шаги

Кадр — одно обновление изображения. Шаг симуляции — одно обновление правил и физики. Они не обязаны совпадать: при 60 кадрах в секунду обычно выполняются два физических шага по 1/120 секунды на кадр.

`dormgame/timing.py` — строки 68–83

```
68     def advance(self, frame_dt: float, update: Callable[[float], object],
69                 paused: bool = False) -> int:
70         self.steps = 0
71         if paused:
72             self.clear()
73             return 0
74         self.accumulator += max(0.0, min(frame_dt, MAX_FRAME_TIME))
75         while self.accumulator + 1e-12 >= self.dt and self.steps < self.max_steps:
76             self.accumulator -= self.dt
77             self.steps += 1
78             if update(self.dt) is False:
79                 self.accumulator = 0.0
80                 break
81         if self.steps == self.max_steps:
82             self.accumulator = min(self.accumulator, self.dt)
83         return self.steps
```

`accumulator` — накопленные секунды, которые ещё не потрачены на симуляцию. Пока их хватает на `self.dt`, вызывается `update`. Из накопителя вычитается один шаг. Предел `max_steps` не позволяет зависанию породить бесконечное догоняние.

В паузе аккумулятор очищается. Сеанс не получает шаги, поэтому не меняются игровое время, положения врагов и защита. При продолжении старое время меню не превращается в рывок.

Числа: `FIXED_DT` = 1/120 секунды; `MAX_STEPS` = 12; `MAX_FRAME_TIME` = 0,1 секунды. Сильное зависание сознательно замедляет игровое время относительно часов на стене.

9 Прыжок который удобно нажимать

Параметр	Значение	Значение для игрока
RUN_SPEED	150 px/s	Предельная скорость бега
GRAVITY	850 px/s ²	Ускорение вниз
JUMP_SPEED	310 px/s	Начальная скорость вверх
MAX_FALL_SPEED	440 px/s	Ограничение падения
COYOTE_TIME	0,105 s	Запас после схода с края
JUMP_BUFFER	0,105 s	Запас нажатия до приземления

dormgame/physics.py — строки 209–215

```
209 def _jump(player: PlayerBody, events: list[str]) -> None:
210     player.vy = -config.JUMP_SPEED
211     player.grounded = False
212     player.support_id = None
213     player.coyote_timer = 0.0
214     player.jump_buffer_timer = 0.0
215     events.append("jump")
```

Скорость вверх отрицательная, потому что у растёт вниз. `_jump` снимает `grounded` и `support_id`: герой уже не должен ехать вместе с платформой. Таймеры обнуляются, потому что разрешение прыжка потрачено.

dormgame/physics.py — строки 293–297

```
293     if not commands.jump_held and player.vy < -config.JUMP_CUT_SPEED:
294         player.vy = -config.JUMP_CUT_SPEED
295     player.vy = min(player.vy + config.GRAVITY * dt, config.MAX_FALL_SPEED)
296     move_body(player, player.vx * dt, player.vy * dt, grid, [p.box for p in platforms])
297     player.grounded, player.support_id = _support(player, grid, platforms)
```

Раннее отпускание ограничивает скорость вверх величиной `JUMP_CUT_SPEED`. Затем гравитация меняет скорость, а скорость, умноженная на `dt`, даёт смещение. Если просто держать пробел, новых фронтов нет и прыжок не повторится.

Полный прыжок в реальной симуляции поднимает героя примерно на 55,24 пикселя и переносит на 108,75 пикселя с разбега. Обычные ступени поднимаются на 32 пикселя. Это запас для честного прохождения, а не попытка использовать предельную высоту.

10 Столкновения с полом и стенами

AABB означает прямоугольник без поворота. Для пересечения должны перекрываться и горизонтальные, и вертикальные промежутки. Координаты остаются дробными; округление происходит только при рисовании.

`dormgame/geometry.py` — строки 33–35

```
33     def intersects(self, other: AABB) -> bool:
34         return (self.left < other.right and self.right > other.left
35                 and self.top < other.bottom and self.bottom > other.top)
```

Одной проверки в конечной точке мало. Если тело быстро прошло тонкую плиту, в конце оно уже ниже неё и пересечения нет. Поэтому физика рассматривает весь промежуток движения по оси и выбирает ближайшую преграждающую поверхность.

`dormgame/physics.py` — строки 174–187

```
174     if abs(dy) > EPSILON:
175         body.grounded = False
176     steps = max(1, ceil(max(abs(dx), abs(dy)) / config.COLLISION_STEP))
177     for _ in range(steps):
178         if _move_axis(body, dx / steps, True, grid, obstacles):
179             result.hit_x = True
180         if _move_axis(body, dy / steps, False, grid, obstacles):
181             result.hit_y = True
182             if dy > 0:
183                 result.landed = True
184                 body.grounded = True
185             else:
186                 result.ceiling = True
187     return result
```

Сначала X: ограничиваем движение стеной. Потом Y: ограничиваем полом или потолком. Всё смещение делится на части до 3 пикселей, а `_move_axis` дополнительно проверяет пересечённый промежуток. При контакте обнуляется только скорость по заблокированной оси.

`grounded` заново проверяется маленьким запросом под ногами. Если платформа осталась позади, герой больше не стоит на ней. Удар головой прекращает подъём, затем действует гравитация.

11 Пространственная сетка и грузовые платформы

Для каждого движения не нужно читать все стены карты. SpatialGrid заранее раскладывает номера стен по клеткам 24×24. Запрос проверяет только клетки рядом с движущимся телом.

dormgame/physics.py — строки 48–56

```
48     def query(self, box: AABB) -> list[AABB]:
49         indices: set[int] = set()
50         candidates: list[AABB] = []
51         for cell in self._occupied_cells(box):
52             for index in self._cells.get(cell, ()):
53                 if index not in indices:
54                     indices.add(index)
55                     candidates.append(self.solids[index])
56         return candidates
```

Одна большая стена может встретиться в нескольких клетках. indices — множество уже взятых номеров, поэтому в результате стены не дублируются. Это отбор кандидатов: точную геометрию дальше проверяет физика.

Перенос на движущейся опоре

dormgame/physics.py — строки 270–282

```
270     carried_id = player.support_id if player.grounded else None
271     if carried_id is not None:
272         carrier = next((p for p in platforms if p.id == carried_id), None)
273         if carrier is not None:
274             old_x, old_y = player.box.x, player.box.y
275             move_body(player, carrier.dx, carrier.dy, grid,
276                     [p.box for p in platforms if p.id != carried_id])
277             if (abs(player.box.x - old_x - carrier.dx) > GROUND_PROBE
278                 or abs(player.box.y - old_y - carrier.dy) > GROUND_PROBE):
279                 return events + ["crushed"]
280
281     for platform in platforms:
282         if platform.id == carried_id:
```

Мир сначала передвигает платформу. Герой знает ID опоры и получает её dx, dy. Перенос также проверяется на столкновения. Если платформе некуда сдвинуть героя из-за стены или потолка, возвращается crushed и модель возрождает его.

Дрон проходит через грузовую решётку по правилу игры. Для него она отсутствует и в навигации, и в физической проверке. Наземные охранники ходят по отведённым статическим участкам.

12 Наследование на примере двух врагов

Enemy описывает общее: ID, координаты, возраст, направление и сброс. Абстрактный `update_behavior` требует от подкласса своё поведение. Так нельзя случайно создать «врага вообще», у которого нет алгоритма движения.

dormgame/enemies.py — строки 32–37

```
32     def update(self, dt: float, player_box: AABB,
33                 grid: SpatialGrid) -> list[str]:
34         if dt <= 0:
35             return []
36         self.age += dt
37         return self.update_behavior(dt, player_box, grid)
```

dormgame/enemies.py — строки 39–42

```
39     @abstractmethod
40     def update_behavior(self, dt: float, player_box: AABB,
41                         grid: SpatialGrid) -> list[str]:
42         """Обновить поведение и вернуть события для представления."""
```

Перед `update_behavior` в исходнике стоит `@abstractmethod`. `PatrolEnemy` и `DroneEnemy` реализуют его по-разному. Первый ходит и падает, второй летит по маршруту. Поэтому гравитация не находится в базовом `Enemy`.

dormgame/model.py — строки 151–153

```
151         for enemy in level.enemies:
152             for kind in enemy.update(dt, level.player.box, level.grid):
153                 self._emit(kind, enemy.box)
```

Это полиморфизм: общий цикл вызывает `enemy.update`, а вызов `update_behavior` попадает в реализацию конкретного класса. Фабрика `build_enemy` выбирает тип только при создании из JSON; цикл не перебирает условия для каждого типа врага.

Понятие	Пример в проекте
Наследование	<code>PatrolEnemy(Enemy)</code> , <code>DroneEnemy(Enemy)</code>
Абстракция	Обязательный <code>update_behavior</code>
Инкапсуляция	Внутренние таймеры, очередь ввода и фаза платформы меняются через методы
Композиция	<code>PatrolEnemy</code> содержит <code>Body</code>
Полиморфизм	Один вызов <code>update</code> для разных подклассов

13 А звезда на маленьком примере

A* произносится «эй стар» или «а звезда». Он ищет кратчайший путь в графе. В этой игре вершина графа — свободная клетка, ребро — разрешённый переход в соседнюю клетку. Диагональных переходов нет.

y / x	0	1	2	3
0	S	.	#	G
1	.	.	#	.
2	.	.	.	.

S — старт (0, 0), G — цель (3, 0), # — стена. Без стены достаточно трёх шагов вправо. Со стеной нужно спуститься на две клетки, пройти вправо и подняться: кратчайший путь имеет семь переходов.

Символ	Смысл	Пример
g	Сколько уже прошли от старта	После одного шага $g = 1$
h	Оценка оставшегося расстояния	Для (1,0) до (3,0): $h = 2$
f	Приоритет в очереди, $g + h$	Для этого соседа: $f = 3$

dormgame/pathfinding.py — строки 37–38

```
37 def manhattan(a: Cell, b: Cell) -> int:
38     return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

Очередь `heapq` извлекает запись с наименьшим приоритетом. `h` направляет поиск к цели, но не знает обо всех обходах. `g` хранит уже найденную реальную стоимость. Если найден более короткий подход к клетке, `g` обновляется.

Манхэттенская оценка не завышает путь по четырём направлениям. Стены могут добавить шаги, но не позволяют добраться быстрее, чем сумма разниц по X и Y. Именно это помогает сохранить кратчайший результат.

14 Настоящий код поиска А звезда

dormgame/pathfinding.py — строки 48–78

```
48     if not grid.is_walkable(start) or not grid.is_walkable(goal):
49         return SearchResult([], 0)
50     serial = itertools.count()
51     queue: list[tuple[int, int, int, Cell]] = [
52         (manhattan(start, goal), 0, next(serial), start)
53     ]
54     g_score: dict[Cell, int] = {start: 0}
55     came_from: dict[Cell, Cell] = {}
56     expanded = 0
57     while queue:
58         _, queued_g, _, current = heapq.heappop(queue)
59         if queued_g != g_score.get(current):
60             continue
61         expanded += 1
62         if current == goal:
63             path = [current]
64             while current in came_from:
65                 current = came_from[current]
66                 path.append(current)
67             path.reverse()
68             return SearchResult(path, expanded)
69         for neighbor in grid.neighbors(current):
70             candidate_g = queued_g + 1
71             if candidate_g < g_score.get(neighbor, math.inf):
72                 came_from[neighbor] = current
73                 g_score[neighbor] = candidate_g
74                 heapq.heappush(queue, (
75                     candidate_g + manhattan(neighbor, goal), candidate_g,
76                     next(serial), neighbor,
77                 ))
78     return SearchResult([], expanded)
```

Читать сверху вниз: проверка входа → очередь и словари → выбор лучшей записи → пропуск устаревшего g → восстановление при достижении цели → улучшение соседей. `came_from` хранит предыдущую клетку. По нему маршрут собирается с конца и разворачивается.

В `heapq` нет отдельной операции уменьшения приоритета. Вместо удаления старой записи добавляется новая, а старая позже пропускается. Пустой список означает отсутствие пути; одна клетка означает совпавшие допустимые старт и цель.

15 Почему дрон обходит стену



Рисунок 3. F3 показывает маршрут под стеной, клеточную сетку, состояние дрона и раскрытые узлы.

Навигация использует клетки 16×16, а корпус дрона имеет размер 18×14. Свободного центра недостаточно: проверяется место для всего корпуса и объём ребра между соседними центрами. Поэтому дрон не режет угол и не пролезает в слишком узкую щель.

Состояние	Когда используется
Патруль	Дрон движется между точками маршрута
Погоня	Игрок приблизился на 180 пикселей или ближе
Возвращение	Игрок оторвался на 245 пикселей; дрон возвращается к патрулю

Путь пересчитывается примерно раз в 0,45 секунды, при смене состояния или заметном смещении цели. Если клетка игрока недоступна, выбирается ближайшая точка в компоненте дрона. Компонента — область клеток, между которыми существует путь.

Дрон замечает героя по расстоянию, в том числе через стену. Это правило обнаружения, а не разрешение проходить через геометрию. Скорость дрона 65 px/s, героя 150 px/s: можно убежать.

16 Страницы бонусы и контрольные точки

dormgame/model.py — строки 169–178

```
169         for page in level.pages:
170             if not page.collected and box.intersects(page.box):
171                 page.collected = True
172                 self._emit('page', page.box, page.text or 'Курсовая намокла, зато теперь с
водой.')
```

```
173         for souvenir in level.souvenirs:
174             if not souvenir.collected and box.intersects(souvenir.box):
175                 souvenir.collected = True
176                 self._emit('souvenir', souvenir.box, souvenir.text)
177         for checkpoint in level.checkpoints:
178             if box.intersects(checkpoint.box) and level.checkpoint_id != checkpoint.id:
```

Проверяются два условия: предмет ещё не собран и герой пересекает его. Затем collected становится True. На следующем шаге первое условие уже ложно, поэтому счёт не увеличивается повторно. Для бонуса используется тот же простой принцип.

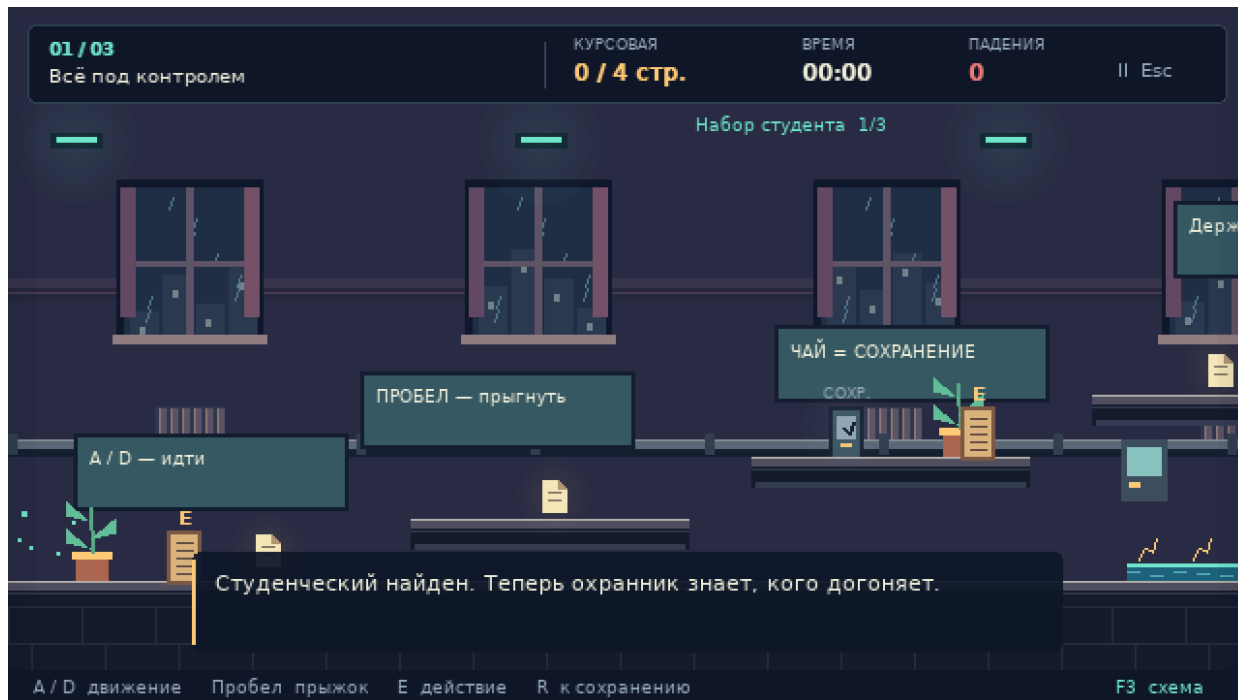


Рисунок 4. Подбор билета добавил один предмет в набор студента и вызвал сообщение.

Контрольная точка меняет spawn. При смерти LevelState.respawn создаёт нового героя, сбрасывает врагов и платформы, но не очищает собранное. Перед переходом между этажами GameSession сохраняет количество страниц и бонусов прошлых карт.

17 От модели к картинке и звуку

Pygame рисует внутреннюю поверхность 640×360, затем увеличивает её до окна. При другом соотношении сторон появляются полосы. Координаты мыши пересчитываются обратно, поэтому кнопки совпадают с картинкой.

dormgame/presentation.py — строки 227–228

```
227     def point(self, x: float, y: float) -> tuple[int, int]:
228         return round(x - self.offset_x), round(y - self.offset_y)
```

Камера — смещение взгляда. Из мировой позиции вычитается её положение. Например, мир $x = 700$ и камера $x = 400$ дают экран $x = 300$. Модельный герой по-прежнему остаётся в 700: камера его не перемещает.

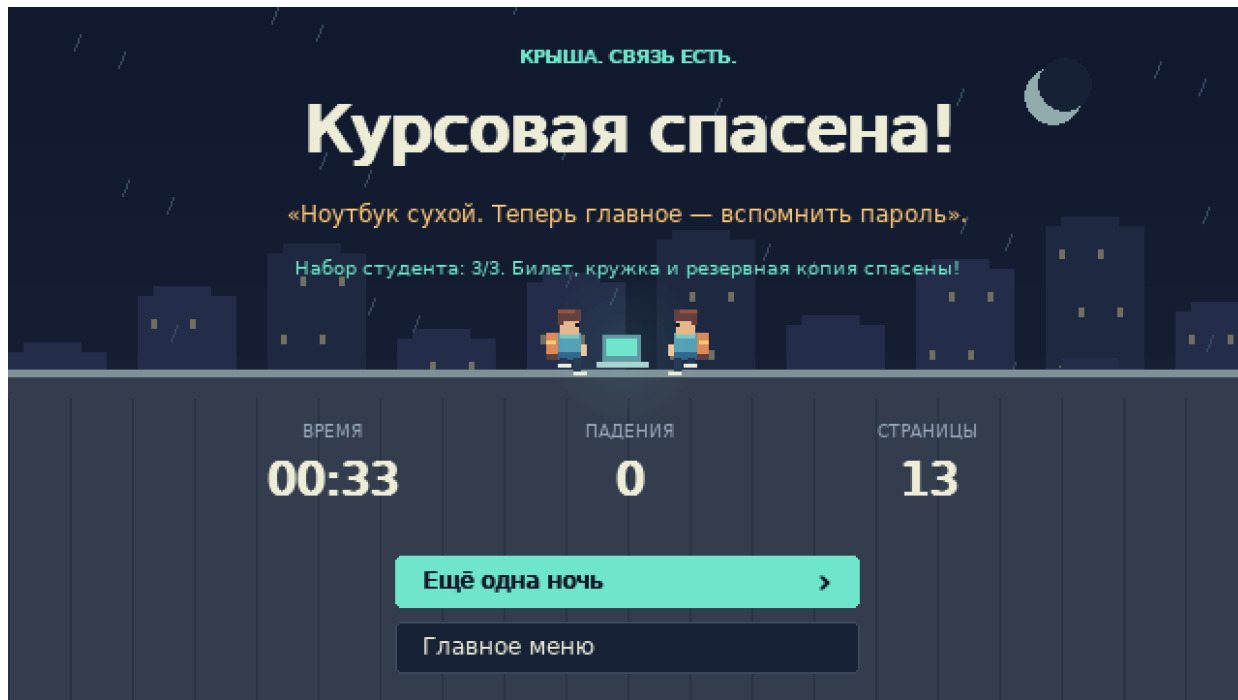


Рисунок 5. Финал автоматического прохождения: 13 страниц, 3 сувенира и 0 смертей.

Спрайты, шрифты и свечения кэшируются. Частиц не больше 90, а при уменьшенных эффектах остаётся до 20. Audio создаёт короткие тоны и собственный луп в памяти; при отсутствии устройства звук пропускается. Настройки хранятся в папке пользователя.

18 Проверки и честные результаты

Текущая версия прошла 75 автоматических тестов. Чистые тесты запускаются даже с -S, который отключает сторонние пакеты. Это показывает, что правила, физика и AI не требуют Pygame.

```
.\.venv\Scripts\python.exe -S -m unittest discover -v
```

Проверка	Что она доказывает
Физика	Контакты с полом, потолком и стенами; тонкая плита; прыжки и движущиеся опоры
A* против BFS	Длина пути совпадает с независимым алгоритмом на небольших картах
Модель	Предметы не дублируются, смерть сохраняет собранное, выход требует страницы
Маршрут	Обычные команды могут провести героя через три настоящих уровня
Pygame	Меню, ввод, пауза, смена экранов и перезапуск работают вместе

```
.\.venv\Scripts\python.exe -m tools.verify_routes
```

```
.\.venv\Scripts\python.exe -m tools.verify_pygame --native
```

Прогон Pygame с бонусами завершился за 33,367 секунды игрового времени: 13 страниц, 3 сувенира, 0 смертей и 20 прыжков. Это точный контроллер, а не оценка времени прохождения человеком.

Человек не проходил игру вручную в рамках этих проверок. Сохранённые кадры просмотрены, а окно Windows запускалось. Не стоит говорить преподавателю, что автоматический прогон доказывает субъективное удобство для всех игроков.

19 Что показать преподавателю

Держите рядом игру и редактор кода. Начните с короткой речи из второго документа. Затем покажите пять действий, каждое связано с конкретной частью реализации.

Шаг	Действие	Объяснение
1	Открыть первый этаж, пройти влево и подобрать билет	Необязательный предмет и событие модели
2	Прыгнуть коротко, затем удерживать пробел	Управляемая высота и скорость вверх
3	Добраться до сохранения и нажать R	Новый герой в checkpoint, страницы остаются
4	Открыть второй этаж и включить F3 возле стены	A* ищет обход для корпуса дрона
5	Открыть Enemy.update и GameSession.update	Один контракт, разные реализации поведения

```
.\.venv\Scripts\python.exe main.py --level 2 --debug
```

Быстрый запуск открывает второй этаж для демонстрации. До решётки нужно пройти по первым ступеням. Стена находится справа от неё. По F3 видны розовые коллайдеры, жёлтый путь, состояние дрона и число раскрытий.

На сцене A* не нужно выигрывать гонку с дроном: достаточно увидеть обход и открыть паузу. По Escape таймеры останавливаются, можно спокойно объяснить рисунок. В коде найдите astar по Ctrl+F и покажите g_score, очередь и came_from.

Если запутались в названии, объясните действие обычными словами и покажите код. Например: «Здесь хранится предыдущая клетка, чтобы потом собрать маршрут». Точность полезнее большого количества терминов.

20 Вопросы преподавателя и ответы

Что здесь делает Pygame

Создаёт окно, читает клавиши, рисует изображения и воспроизводит звуки. Правила, физика и A* написаны отдельно.

Где наследование

Enemy — базовый абстрактный класс. PatrolEnemy и DroneEnemy наследуют общий жизненный цикл и задают разное поведение.

Где полиморфизм

В GameSession.update у каждого врага вызывается enemy.update. Какая реализация update_behavior работает, зависит от класса объекта.

Что такое self

Ссылка на конкретный объект. self.box — прямоугольник именно этого героя, врага или предмета.

Что такое AABB

Прямоугольник без поворота. У него есть x, y, ширина и высота; столкновение определяется по перекрытию осей.

Почему герой не пролетает сквозь пол

Проверяется весь путь по оси до ближайшей поверхности, а движение дополнительно делится на отрезки до 3 пикселей.

Что такое g h и f в A*

g — уже пройденная стоимость; h — оценка оставшегося пути; $f = g + h$. Очередь выбирает наименьший f.

Почему манхэттенская эвристика

Разрешены четыре осевых направления с ценой 1. Без стен сумма разниц координат даёт кратчайшее число шагов и не превышает оценку.

Почему нельзя брать только ближайшую клетку игрока

Она может быть слишком узкой или отделённой стеной. Выбирается ближайшая точка в той же компоненте связности, что и дрон.

21 Вопросы преподавателя и ответы

Что происходит при отсутствии пути

A* возвращает пустой список. Дрон не летит сквозь стену и повторяет поиск по интервалу.

Какая сложность A*

$O((V + E) \log V)$, здесь E не больше $4V$, поэтому $O(V \log V)$. V — свободные клетки, E — переходы между ними.

Зачем фиксированный шаг

Физика всегда получает одинаковую длительность обновления. Количество кадров отрисовки может меняться, а шаг симуляции остаётся $1/120$ секунды.

Что такое coyote time и jump buffer

Первое позволяет прыгнуть примерно 0,1 секунды после ухода с края. Второе запоминает нажатие примерно 0,1 секунды до приземления.

Что сохраняется после смерти

Страницы, сувениры и текущая контрольная точка. Враги и платформы сбрасываются, герой получает короткую защиту.

Нужно ли собирать сувениры для выхода

Нет. Выход требует только страницы. Билет, кружка и флешка дают дополнительный результат в финале.

Почему дрон проходит через движущуюся решётку

Таково правило: решётка держит героя, а маленький дрон пролетает через неё. И поиск, и фактическое движение дрона учитывают только статические стены.

Как проверена проходимость

Контроллер использует обычные команды и доходит до финала. Есть отдельный прогон через окно Pугame. Это автоматическое, а не ручное прохождение.

Что можно улучшить

Сохранение прохождения между запусками, больше ручных карт и более подробные анимации. Сейчас между запусками сохраняются только настройки.

22 Небольшая практика перед защитой

Выполняйте упражнения в копии папки проекта. После изменения запустите игру, а затем тесты. Перед самой сдачей вернитесь к проверенной версии.

Объяснить один предмет

Откройте `GameSession.update`. Найдите `collected = True`. Объясните, почему следующая проверка не засчитает предмет снова. Затем найдите событие `souvenir` в `Audio` и `Renderer`.

Предсказать изменение скорости

Найдите `RUN_SPEED` в `config.py`. Увеличение меняет максимальную скорость, но не делает её мгновенной: разгон задаёт `ACCELERATION`. Скажите, какие тесты и прыжки нужно проверить после такого изменения.

Изменить одну реплику

В `data/level1.json` найдите `text` у страницы. Измените только текст. Геометрия, ID и условие выхода останутся прежними. Это пример разделения данных и поведения.

Нарисовать путь на бумаге

Повторите маленькую карту из главы об `A*`. Подпишите старт, цель, стену и один путь из семи переходов. Объясните, почему прямой путь закрыт.

Прочитать метод вслух

Выберите `AABB.intersects` или `GameSession._die`. Для каждой строки назовите входные данные, действие и результат. После этого объясните метод без чтения.

Самопроверка: можете ли вы за минуту объяснить, где координаты, где рисунок, почему нужен `dt` и чем очередь событий отличается от очереди поиска `A*`? Если нет, вернитесь к соответствующей главе, а не к заучиванию всей речи.

23 Критерии оценки и ограничения

Критерий	Вес	Что показать
DRY	2	Общие AABB, move_body, Enemy.update и кэши
KISS	2	Короткие ручные карты и простые структуры
SOLID	2	Разделение модулей и общий контракт врагов
Качество кода	1	Именованные параметры, типы и проверка данных
Инкапсуляция абстракция наследование	3	Enemy, внутренние таймеры и два наследника
Сложность алгоритма	до 15	Самостоятельный A*, heapq, g-score и BFS-тесты
Разделение ответственности	10	Модель, физика, AI и представление
Структура проекта	5	Пакет, данные, тесты и документы
Работоспособность	3	Три этажа, финал и новая игра
Удобство	2	Буфер прыжка, coyote time, пауза и сохранения
Сюжет оформление музыка	6	Ночная общага, реплики, сувениры и локальный луп

Вес критерия взят из задания. Это не обещание полученной оценки: на защите важны работающая демонстрация и понимание реализации.

Ограничения: нет клонов, поворачиваемых коллайдеров, редактора карт и сохранения прохождения между запусками. Настройки сохраняются. Дрон видит по расстоянию и игнорирует грузовые решётки по правилам мира. A* находит кратчайший путь в своей сетке, а не произвольную идеальную траекторию в непрерывном пространстве.

24 Оценка сложности без завышенных обещаний

V — допустимые клетки навигации, E — переходы, C — все клетки, S — статические прямоугольники, L — длина пути. Для сетки с четырьмя направлениями E не больше $4V$.

Операция	Время	Почему
A^*	$O((V + E) \log V)$	Очередь приоритетов обрабатывает вершины и улучшения рёбер
A^* на этой сетке	$O(V \log V)$	Число рёбер пропорционально числу вершин
Восстановление	$O(L)$	Проходим по <code>came_from</code> и разворачиваем список
Построение навигации	$O(C \cdot S + V \log V)$	Однократные проверки стен и сортировка свободных клеток
Компоненты	$O(V + E)$	Обход связей графа
Точка приближения	$O(V_k)$ в худшем случае	Просмотр компоненты дрона, если клетка игрока недоступна
Запрос <code>spatial grid</code>	$O(Q + K)$ в среднем	Q посещённых клеток и K прочитанных ссылок, включая повторы

Память A^* — $O(V + E)$, здесь $O(V)$. Словарь и множество считаются имеющими среднюю постоянную стоимость отдельной операции. Но весь `spatial grid`-запрос не $O(1)$: он зависит от количества затронутых клеток и ссылок.

В движении дрона путь хранится коротким списком; удаление первой клетки стоит $O(L)$. Это стоимость следования пути, а не самого алгоритма поиска. Для текущих небольших карт решение достаточно, а на длинных маршрутах можно использовать очередь или индекс.

25 Полный справочник функций и методов игры

Ниже перечислены все 137 явно объявленные функции и методы пакета `dormgame`, включая внутренние методы, свойства и вложенные помощники валидатора. Список извлечён из Python-кода автоматически; пояснения написаны отдельно. Сборщик проверяет, что у каждого объявления есть описание.

Файлы `config.py` и `commands.py` содержат параметры и `dataclass` без собственных методов. `__init__.py` объявляет пакет. `main.py` передаёт управление `app.main`. Автоматически создаваемые `dataclass` методы не имеют строк в исходном файле и поэтому не выдаются за написанные вручную. Тестовые и служебные скрипты разобраны по назначению в главе о проверках; они не входят в игровой пакет.

Сигнатура показывает имя и параметры. После стрелки указан ожидаемый тип результата. Если метод возвращает `None`, он обычно изменяет свой объект или выполняет действие. Фрагменты сигнатур ниже взяты из текущих файлов; многоточие в длинном объявлении означает только перенос записи в справочнике, а не пропуск реализации в проекте.

Модуль `geometry`

Границы и пересечения прямоугольников без графической библиотеки.

`AABB.left`

`dormgame/geometry.py` | строки 14–15

```
def left(self) -> float:
```

Возвращает левую границу прямоугольника. Это та же координата `x`; свойство читается как `box.left` без скобок.

`AABB.right`

`dormgame/geometry.py` | строки 18–19

```
def right(self) -> float:
```

Складывает `x` и ширину `w`. Полученная правая граница нужна, чтобы остановить тело перед левой стороной стены.

`AABB.top`

`dormgame/geometry.py` | строки 22–23

```
def top(self) -> float:
```

Возвращает верхнюю границу `y`. В экранных координатах меньший `y` означает положение выше.

`AABB.bottom`

`dormgame/geometry.py` | строки 26–27

```
def bottom(self) -> float:
```

Складывает `y` и высоту `h`. Именно нижняя граница героя должна совпасть с верхом пола при приземлении.

`AABB.center`

`dormgame/geometry.py` | строки 30–31

```
def center(self) -> tuple[float, float]:
```

Возвращает пару координат центра. Центром пользуются камера, расстояние обнаружения дрона и графические эффекты.

AABB.intersects

dormgame/geometry.py | строки 33–35

```
def intersects(self, other: AABB) -> bool:
```

Проверяет перекрытие прямоугольников по обеим осям. Простое касание краями не считается пересечением, поэтому стояние на полу не означает проникновение в него.

AABB.moved

dormgame/geometry.py | строки 37–38

```
def moved(self, dx: float, dy: float) -> AABB:
```

Создаёт новый прямоугольник со смещением dx и dy. Исходный объект остаётся на месте; так можно проверить будущую позицию до движения.

AABB.expanded

dormgame/geometry.py | строки 40–42

```
def expanded(self, margin: float) -> AABB:
```

Создаёт прямоугольник с запасом со всех сторон. Например, взаимодействовать с запиской разрешается немного раньше непосредственного касания.

AABB.copy

dormgame/geometry.py | строки 44–45

```
def copy(self) -> AABB:
```

Возвращает отдельную копию координат и размеров. Изменение копии не передвигает исходный объект.

Модуль physics

Как тела двигаются, прыгают, останавливаются о стены и едут на платформах.

SpatialGrid.__init__

dormgame/physics.py | строки 28–36

```
def __init__(self, solids: list[AABB], cell_size: float = config.TILE_SIZE):
```

Размещает номера неподвижных стен в клетках пространственной сетки. Большая стена входит в несколько клеток; индекс строится один раз при загрузке карты.

SpatialGrid._occupied_cells

dormgame/physics.py | строки 38–46

```
def _occupied_cells(self, box: AABB):
```

Перебирает координаты клеток, затронутых прямоугольником. `yield` возвращает их по одной; отрицательные координаты также обрабатываются правильно.

SpatialGrid.query

dormgame/physics.py | строки 48–56

```
def query(self, box: AABB) -> list[AABB]:
```

Собирает ближайшие стены из затронутых клеток и удаляет повторные номера через `set`. Результат — кандидаты: точное пересечение проверяется после запроса.

MovingPlatform.__init__

dormgame/physics.py | строки 90–100

```
def __init__(self, id: str, start: AABB, end: tuple[float, float], speed: float):
```

Запоминает начальный прямоугольник, конечную точку и скорость. Копия start нужна для предсказуемого возврата после смерти.

MovingPlatform.reset

dormgame/physics.py | строки 102–105

```
def reset(self) -> None:
```

Возвращает платформу к старту и обнуляет фазу движения и перенос dx, dy. Герой не получит смещение от предыдущей попытки.

MovingPlatform.update

dormgame/physics.py | строки 107–117

```
def update(self, dt: float) -> None:
```

Продвигает платформу по отрезку туда и обратно. Вычисляет разницу между прошлой и новой позицией, чтобы перенести стоящего на ней героя.

_overlap_x

dormgame/physics.py | строки 120–121

```
def _overlap_x(a: AABB, b: AABB) -> bool:
```

Проверяет перекрытие горизонтальных проекций с малым допуском EPSILON. Используется при столкновениях с полом, потолком и опорой.

_overlap_y

dormgame/physics.py | строки 124–125

```
def _overlap_y(a: AABB, b: AABB) -> bool:
```

Проверяет перекрытие вертикальных проекций. При движении вбок препятствие важно только тогда, когда тело находится с ним на одной высоте.

_move_axis

dormgame/physics.py | строки 128–162

```
def _move_axis(body: Body, distance: float, horizontal: bool, grid: SpatialGrid, extra: list[AABB]) -> bool:
```

Для одной оси находит ближайшую преграждающую поверхность во всём промежутке движения. Ограничивает смещение и обнуляет скорость по заблокированной оси.

move_body

dormgame/physics.py | строки 165–187

```
def move_body(body: Body, dx: float, dy: float, grid: SpatialGrid, extra: list[AABB] | None = None) -> CollisionResult:
```

Разбивает движение на подшаги до 3 пикселей, на каждом разрешает X, затем Y. Возвращает признаки контакта со стеной, полом и потолком.

_support

dormgame/physics.py | строки 190–202

```
def _support(body: Body, grid: SpatialGrid, platforms: list[MovingPlatform], previous: bool = False) -> tuple[bool, str | None]:
```

Проверяет наличие опоры прямо под ногами. Возвращает grounded и ID платформы; при подъёме героя опора не назначается.

`_approach`

dormgame/physics.py | строки 205–206

```
def _approach(value: float, target: float, amount: float) -> float:
```

Приближает число к целевому значению на ограниченную величину. Так скорость растёт и уменьшается плавно, не перескакивая через цель.

`_jump`

dormgame/physics.py | строки 209–215

```
def _jump(player: PlayerBody, events: list[str]) -> None:
```

Задаёт скорость вверх, снимает опору и очищает таймеры разрешения прыжка. Добавляет событие `jump` для звука и представления.

`_platform_push`

dormgame/physics.py | строки 218–243

```
def _platform_push(player: PlayerBody, platform: MovingPlatform, grid: SpatialGrid, others: list[AABB]) -> bool:
```

Проверяет, не наехала ли движущаяся платформа на героя, и пробует безопасно сдвинуть его. Если стена мешает, возвращает признак зажатия.

`update_player`

dormgame/physics.py | строки 246–307

```
def update_player(player: PlayerBody, commands: Commands, dt: float, grid: SpatialGrid, platforms: list[MovingPlatform]) -> list[str]:
```

Выполняет весь физический шаг героя: ввод и таймеры, прыжок, перенос опорой, ускорение, гравитация, коллизии и новое определение опоры. Возвращает события, а не рисует кадр.

Модуль `pathfinding`

Какие клетки доступны дрону и как найти кратчайший путь между ними.

`SearchGrid.is_walkable`

dormgame/pathfinding.py | строки 22–22

```
def is_walkable(self, cell: Cell) -> bool: ...
```

Объявляет контракт графа для A^* : можно ли занимать указанную клетку. Это описание интерфейса `Protocol`; конкретная сетка предоставляет реализацию.

`SearchGrid.neighbors`

dormgame/pathfinding.py | строки 24–24

```
def neighbors(self, cell: Cell) -> Iterable[Cell]: ...
```

Объявляет второй метод контракта: получить допустимых соседей вершины. Благодаря этому A^* можно проверять на маленьких тестовых графах.

`SearchResult.cost`

dormgame/pathfinding.py | строки 33–34

```
def cost(self) -> int | None:
```

Возвращает количество переходов: число клеток пути минус один. Для пустого пути возвращает `None`, поскольку путь не найден.

manhattan

dormgame/pathfinding.py | строки 37–38

```
def manhattan(a: Cell, b: Cell) -> int:
```

Складывает разницы координат по модулю. Оценивает оставшееся число шагов при движении только вверх, вниз, влево и вправо.

astar

dormgame/pathfinding.py | строки 41–78

```
def astar(grid: SearchGrid, start: Cell, goal: Cell) -> SearchResult:
```

Ищет кратчайший путь с помощью `heapq`, `g_score` и эвристики. Проверяет устаревшие записи очереди и восстанавливает маршрут через `came_from`.

NavigationGrid.__init__

dormgame/pathfinding.py | строки 90–136

```
def __init__(self, width: int, height: int, solids: list[AABB], cell_size: int = 16, agent_size: tuple[float, float] = (18, 14)):
```

Строит свободные клетки и безопасные рёбра с учётом корпуса дрона. Затем объединяет связанные клетки в компоненты, чтобы выбирать достижимую цель.

NavigationGrid.cell_center

dormgame/pathfinding.py | строки 138–140

```
def cell_center(self, cell: Cell) -> tuple[float, float]:
```

Переводит номер клетки в мировые координаты её центра. Например, клетка 0 по X имеет центр на половине размера клетки.

NavigationGrid.world_to_cell

dormgame/pathfinding.py | строки 142–144

```
def world_to_cell(self, point: tuple[float, float]) -> Cell:
```

Делит мировые координаты на размер клетки и округляет вниз. Это обратный переход от позиции сущности к адресам навигационной сетки.

NavigationGrid.agent_box

dormgame/pathfinding.py | строки 146–148

```
def agent_box(self, point: tuple[float, float]) -> AABB:
```

Строит прямоугольник дрона вокруг заданного центра. Проверка размера корпуса защищает от маршрутов через слишком узкие щели.

NavigationGrid.segment_clear

dormgame/pathfinding.py | строки 150–162

```
def segment_clear(self, start: tuple[float, float], end: tuple[float, float]) -> bool:
```

Проверяет весь объём перехода между центрами. Даже если оба центра свободны, тонкая стена между ними делает ребро недопустимым.

NavigationGrid.is_walkable

dormgame/pathfinding.py | строки 164–165

```
def is_walkable(self, cell: Cell) -> bool:
```

Проверяет принадлежность клетки множеству `walkable`. Граф уже построен, поэтому повторно сравнивать клетку со всеми стенами не нужно.

NavigationGrid.neighbors

dormgame/pathfinding.py | строки 167–168

```
def neighbors(self, cell: Cell) -> tuple[Cell, ...]:
```

Возвращает заранее подготовленный кортеж соседних клеток. Для отсутствующей клетки возвращает пустой кортеж.

NavigationGrid.nearest_free

dormgame/pathfinding.py | строки 170–179

```
def nearest_free(self, point: tuple[float, float], component: int | None = None) -> Cell | None:
```

Выбирает свободный центр с наименьшим расстоянием до точки. При равенстве расстояний использует координаты, поэтому выбор воспроизводим.

NavigationGrid.reachable_target

dormgame/pathfinding.py | строки 181–191

```
def reachable_target(self, start: Cell, point: tuple[float, float]) -> Cell | None:
```

Если клетка игрока находится в компоненте дрона, возвращает её. Иначе выбирает ближайшую свободную точку только внутри этой компоненты.

Модуль enemies

Общий контракт врагов и два разных способа движения.

Enemy.__init__

dormgame/enemies.py | строки 24–30

```
def __init__(self, identifier: str, box: AABB):
```

Запоминает ID, прямоугольник, копию стартовой позиции, направление, возраст и начальное состояние. Не добавляет гравитацию всем врагам.

Enemy.update

dormgame/enemies.py | строки 32–37

```
def update(self, dt: float, player_box: AABB, grid: SpatialGrid) -> list[str]:
```

Обновляет общий возраст и вызывает `update_behavior`. Это единая точка входа игрового цикла для разных видов врагов.

Enemy.update_behavior

dormgame/enemies.py | строки 40–42

```
def update_behavior(self, dt: float, player_box: AABB, grid: SpatialGrid) -> list[str]:
```

Обязательный абстрактный метод. Каждый подкласс должен определить собственное поведение, иначе Python не даст создать его экземпляр.

Enemy.reset

dormgame/enemies.py | строки 44–48

```
def reset(self) -> None:
```

Восстанавливает исходную позицию и общие поля врага. Подклассы дополняют этот сброс своими данными.

PatrolEnemy.__init__

dormgame/enemies.py | строки 54–58

```
def __init__(self, identifier: str, box: AABB, left: float, right: float):
```

Задаёт границы наземного патруля и создаёт Body. Враг использует физическое тело как часть себя: это композиция.

PatrolEnemy.reset

dormgame/enemies.py | строки 60–62

```
def reset(self) -> None:
```

Вызывает общий сброс Enemy и заново создаёт Body для восстановленного прямоугольника. Старые скорости не переносятся в новую попытку.

PatrolEnemy.update_behavior

dormgame/enemies.py | строки 64–83

```
def update_behavior(self, dt: float, player_box: AABB, grid: SpatialGrid) -> list[str]:
```

Применяет гравитацию, смотрит вперёд на стену, обрыв и границу участка. Разворачивается и ходит только при наличии опоры.

DroneEnemy.__init__

dormgame/enemies.py | строки 92–107

```
def __init__(self, identifier: str, box: AABB, route: list[tuple[float, float]], solids: list[AABB], width: int, height: int):
```

Строит навигацию для корпуса, привязывает старт к безопасному центру и создаёт поля патруля и поиска. Для дрона наземное Body не нужно.

DroneEnemy.reset

dormgame/enemies.py | строки 109–116

```
def reset(self) -> None:
```

Восстанавливает позицию через Enemy, очищает путь, счётчик раскрытий и таймеры. Следующая попытка начинается с предсказуемого патруля.

DroneEnemy._change_state

dormgame/enemies.py | строки 118–121

```
def _change_state(self, state: str) -> None:
```

Меняет режим и разрешает новый поиск. Текущий безопасный конец ребра сохраняется, чтобы дрон не срезал угол стены.

DroneEnemy.update_behavior

dormgame/enemies.py | строки 124–153

```
def update_behavior(self, dt: float, player_box: AABB, grid: SpatialGrid) -> list[str]:
```

Выбирает патруль, погоню или возвращение по расстоянию до героя. Пересчитывает путь по таймеру или изменению цели и перемещает дрон.

DroneEnemy._find_path

dormgame/enemies.py | строки 155–167

```
def _find_path(self, target: tuple[float, float]) -> None:
```

Определяет безопасный старт, выбирает достижимую цель и вызывает astar. Сохраняет путь и число раскрытых узлов для движения и F3.

DroneEnemy._follow_path

dormgame/enemies.py | строки 169–193

```
def _follow_path(self, distance: float, grid: SpatialGrid) -> None:
```

Расходует разрешённое расстояние полёта на последовательные точки пути. Перед каждым смещением проверяет реальные стены через пространственную сетку.

build_enemy

dormgame/enemies.py | строки 196–207

```
def build_enemy(spec: dict, solids: list[AABB], width: int, height: int) -> Enemy:
```

По kind из JSON один раз создаёт нужный подкласс. Выбор типа находится в фабрике, а последующее обновление использует общий контракт Enemy.

Модуль levels

Как из JSON получить проверенные исходные данные трёх этажей.

_number

dormgame/levels.py | строки 16–17

```
def _number(value: Any) -> bool:
```

Проверяет, что значение — конечное число, а не bool, NaN или бесконечность. Это защищает геометрию от неверных данных.

validate_level

dormgame/levels.py | строки 20–121

```
def validate_level(spec: dict[str, Any]) -> None:
```

Проверяет обязательные поля, типы объектов, границы, уникальные ID, безопасные точки появления и маршруты платформ. Неверная карта отклоняется с объяснением.

validate_level.point

dormgame/levels.py | строки 34–38

```
def point(value: Any, label: str) -> None:
```

Вложенный помощник проверки пары координат. Доступен только внутри validate_level и использует размеры проверяемой карты.

validate_level.rect

dormgame/levels.py | строки 40–46

```
def rect(value: Any, label: str) -> AABB:
```

Вложенный помощник проверяет четыре числа, положительные размеры и нахождение внутри карты. Возвращает проверенный AABB.

validate_level.identify

dormgame/levels.py | строки 48–53

```
def identify(obj: Any, label: str) -> None:
```

Вложенный помощник требует непустой строковый ID и запрещает повторы между группами объектов. Общий набор identifiers принадлежит проверке одной карты.

load_levels

dormgame/levels.py | строки 124–128

`def load_levels(directory: Path = DATA_DIR) -> list[dict[str, Any]]:`
Читает три JSON-файла в UTF-8, проверяет каждый и возвращает список описаний. Путь определяется относительно файлов проекта, а не текущего окна терминала.

Модуль model

Правила сеанса, предметы, смерть, сохранение и переход к следующему этажу.

Item.from_spec

dormgame/model.py | строки 34–37

`def from_spec(cls, spec: dict[str, Any]) -> Item:`
Создаёт предмет из словаря JSON. Преобразует `rect` в AABB и переносит текст, точку появления и вид сувенира.

LevelState.__init__

dormgame/model.py | строки 41–60

`def __init__(self, spec: dict[str, Any]):`
Собирает один этаж из данных: стены, индекс, героя, врагов, платформы, предметы и выход. Здесь появляются изменяемые объекты текущей попытки.

LevelState.collected_pages

dormgame/model.py | строки 63–64

`def collected_pages(self) -> int:`
Считает страницы с флагом `collected`. Значения `True` суммируются как единицы, `False` как нули.

LevelState._new_player

dormgame/model.py | строки 66–67

`def _new_player(self) -> PlayerBody:`
Создаёт новое тело героя в текущей точке `spawn` с размерами из `config`. Старые скорости и таймеры остаются у прежнего тела.

LevelState.respawn

dormgame/model.py | строки 69–77

`def respawn(self) -> None:`
Сбрасывает врагов и платформы, создаёт героя в контрольной точке и выдаёт короткую защиту. Уже собранные страницы и сувениры не очищаются.

GameSession.__init__

dormgame/model.py | строки 82–84

`def __init__(self, specs: list[dict[str, Any]] | None = None):`
Принимает готовые описания карт или загружает стандартные. Затем создаёт новый сеанс через `new_game`.

GameSession.new_game

dormgame/model.py | строки 86–95

`def new_game(self) -> None:`
Обнуляет время, смерти, накопленные страницы и сувениры, очищает события и открывает первый этаж. Это полный перезапуск прогресса.

GameSession.total_pages

dormgame/model.py | строки 98–99

```
def total_pages(self) -> int:
```

Складывает страницы прошлых этажей и текущего. Так HUD и финал получают общую статистику без перебора уже закрытых уровней.

GameSession.total_souvenirs

dormgame/model.py | строки 102–103

```
def total_souvenirs(self) -> int:
```

Складывает сохранённый счёт прошлых этажей и собранные сувениры текущего. Не участвует в условии открытия выхода.

GameSession.souvenir_goal

dormgame/model.py | строки 106–107

```
def souvenir_goal(self) -> int:
```

Считает общее количество необязательных сувениров в описаниях карт. Для стандартной кампании возвращает три.

GameSession.drain_events

dormgame/model.py | строки 109–111

```
def drain_events(self) -> list[GameEvent]:
```

Отдаёт накопленные события и заменяет очередь пустой. Представление получает каждое событие один раз.

GameSession._emit

dormgame/model.py | строки 113–114

```
def _emit(self, kind: str, box: AABB, text: str = '') -> None:
```

Добавляет событие с видом, центром объекта и необязательным текстом. Модель сообщает, что случилось, без вызовов Pugame.

GameSession._die

dormgame/model.py | строки 116–119

```
def _die(self, reason: str) -> None:
```

Отправляет событие смерти, увеличивает счётчик и вызывает respawn. Смерть обрабатывается одним общим способом для разных причин.

GameSession.advance_level

dormgame/model.py | строки 121–131

```
def advance_level(self) -> None:
```

Проверяет завершение этажа. Сохраняет собранное и открывает следующий; после последнего выставляет finished и отправляет win.

GameSession.update

dormgame/model.py | строки 133–196

```
def update(self, dt: float, commands: Commands) -> None:
```

Применяет один шаг правил в фиксированном порядке. Обновляет движение, врагов, смерть, предметы, контрольную точку и E; в паузе сразу выходит.

Модуль timing

Как сохранить ввод и обновлять игру одинаковыми маленькими шагами.

InputBuffer.__init__

dormgame/timing.py | строки 19–25

```
def __init__(self) -> None:
```

Создаёт очередь фронтов прыжка, направления и флаги разовых действий. Физика ещё не потребила ни одного нажатия.

InputBuffer.jump

dormgame/timing.py | строки 27–30

```
def jump(self, pressed: bool) -> None:
```

Добавляет изменение кнопки только при переходе нажато или отпущено. Повторные KEYDOWN при удержании не превращаются в новые прыжки.

InputBuffer.interact

dormgame/timing.py | строки 32–33

```
def interact(self) -> None:
```

Запоминает одно взаимодействие E до следующего шага. Кадр без физического обновления не теряет действие.

InputBuffer.reset_checkpoint

dormgame/timing.py | строки 35–36

```
def reset_checkpoint(self) -> None:
```

Запоминает один запрос возврата R. Защиту от слишком частого возврата затем проверяет модель.

InputBuffer.consume

dormgame/timing.py | строки 38–46

```
def consume(self) -> Commands:
```

Формирует Commands для одного шага, потребляет один фронт прыжка и сбрасывает разовые флаги. Удержание и направление сохраняются отдельно.

InputBuffer.clear

dormgame/timing.py | строки 48–52

```
def clear(self) -> None:
```

Очищает очередь и состояния при смене экрана. Старое нажатие до паузы не сработает неожиданно после продолжения.

FixedStepper.__init__

dormgame/timing.py | строки 58–62

```
def __init__(self, dt: float = FIXED_DT, max_steps: int = MAX_STEPS) -> None:
```

Запоминает фиксированную длительность шага и предел догоняющих шагов. Аккумулятор сначала пуст.

FixedStepper.clear

dormgame/timing.py | строки 64–66

```
def clear(self) -> None:
```

Обнуляет накопленное время и число шагов кадра. Используется при паузе и переходах.

FixedStepper.advance

dormgame/timing.py | строки 68–83

`def advance(self, frame_dt: float, update: Callable[[float], object], paused: bool = False) -> int:`
Добавляет время кадра, вызывает обновление порциями по 1/120 секунды и ограничивает догоняние. При паузе очищает накопление.

Модуль audio

Как локально создаются и воспроизводятся звуки.

Audio.__init__

dormgame/audio.py | строки 13–37

`def __init__(self) -> None:`
Пробует открыть аудиоустройство и один раз создаёт короткие звуки и музыкальный луп. При недоступном устройстве оставляет `available` равным `False`.

Audio._pcm

dormgame/audio.py | строки 40–45

`def _pcm(values: list[float]) -> bytes:`
Преобразует значения звуковой волны в 16-битные стереосэмплы. Ограничивает амплитуду, чтобы не выйти за допустимый диапазон.

Audio._tone

dormgame/audio.py | строки 47–57

`def _tone(self, start: float, end: float, length: float) -> pygame.mixer.Sound:`
Синтезирует короткий тон с изменением частоты и затуханием. Возвращает готовый `pygame.mixer.Sound` для последующего воспроизведения.

Audio._make_music

dormgame/audio.py | строки 59–85

`def _make_music(self) -> pygame.mixer.Sound:`
Создаёт оригинальную спокойную мелодию и бас в памяти. Игра не загружает музыку из Интернета.

Audio.configure

dormgame/audio.py | строки 87–100

`def configure(self, enabled: bool, music_enabled: bool) -> None:`
Применяет настройки звука и музыки. Запускает или останавливает фоновый канал; при выключении звука останавливает эффекты.

Audio.play

dormgame/audio.py | строки 102–106

`def play(self, name: str) -> None:`
Воспроизводит звук события, если он существует и звук разрешён. Без устройства спокойно пропускает действие.

Модуль presentation

Как состояние модели превращается в видимый кадр.

`time_text`

`dormgame/presentation.py` | строки 24–26

```
def time_text(seconds: float) -> str:
```

Переводит секунды в строку минуты и секунды. Например, 75 секунд показываются как 01:15.

`Text.__init__`

`dormgame/presentation.py` | строки 32–41

```
def __init__(self) -> None:
```

Находит комплектный кириллический шрифт или резервный системный. Создаёт кэши размеров шрифта и готовых подписей.

`Text.font`

`dormgame/presentation.py` | строки 43–48

```
def font(self, size: int, bold: bool = False) -> pygame.font.Font:
```

Возвращает шрифт заданного размера и жирности из кэша. Загружает новый объект только для ещё не использованного сочетания.

`Text.draw`

`dormgame/presentation.py` | строки 50–65

```
def draw(self, surface: pygame.Surface, value: str, xy: tuple[float, float], size: int = 12, color: tuple = WHITE, bold: bool = False, center: bool = False) -> pygame.Rect:
```

Рисует подпись и возвращает её экранный прямоугольник. Повторяющийся текст берётся из кэша изображений.

`Text.wrapped`

`dormgame/presentation.py` | строки 67–84

```
def wrapped(self, surface: pygame.Surface, value: str, xy: tuple[int, int], max_width: int, size: int = 12, color: tuple = WHITE, line_height: int | None = None) -> int:
```

Разбивает длинный текст на строки, помещающиеся в заданную ширину. Возвращает нижнюю координату, удобную для дальнейшей вёрстки.

`Sprites.__init__`

`dormgame/presentation.py` | строки 88–89

```
def __init__(self) -> None:
```

Создаёт кэш рисунков персонажей и предметов. Спрайты дальше строятся только по необходимости.

`Sprites.actor`

`dormgame/presentation.py` | строки 91–147

```
def actor(self, kind: str, facing: int, frame: int) -> pygame.Surface:
```

Возвращает рисунок студента, охранника или дрона с нужным направлением и кадром анимации. Готовое сочетание берётся из кэша.

Sprites.object

dormgame/presentation.py | строки 149–182

```
def object(self, kind: str) -> pygame.Surface:
```

Создаёт и кэширует повторяющиеся предметы интерьера: растение, сушилку, ящик, радиатор и устройство.

Renderer.__init__

dormgame/presentation.py | строки 199–217

```
def __init__(self, canvas: pygame.Surface) -> None:
```

Создаёт инструменты рисования, камеру, частицы, затемнение краёв и кэш свечений. Все эти данные относятся к представлению.

Renderer.reset_camera

dormgame/presentation.py | строки 219–225

```
def reset_camera(self, level: Any) -> None:
```

Ставит камеру около героя в границах уровня, очищает частицы и тряску. Используется при входе на новый этаж.

Renderer.point

dormgame/presentation.py | строки 227–228

```
def point(self, x: float, y: float) -> tuple[int, int]:
```

Вычитает смещение камеры из мировых координат и округляет результат. Получает координаты для рисования на экране.

Renderer.rect

dormgame/presentation.py | строки 230–231

```
def rect(self, box: Any) -> pygame.Rect:
```

Преобразует модельный AABB в целочисленный pygame.Rect. Сам AABB при этом не изменяется.

Renderer.glow

dormgame/presentation.py | строки 233–241

```
def glow(self, position: tuple[int, int], color: tuple, radius: int = 25) -> None:
```

Рисует мягкое свечение заданного цвета и размера. Повторно использует сохранённую поверхность свечения.

Renderer.tick

dormgame/presentation.py | строки 243–254

```
def tick(self, dt: float, settings: Any) -> None:
```

Продвигает время анимаций, сообщения, тряску и частицы. Удаляет погасшие частицы и соблюдает режим уменьшенных эффектов.

Renderer.accept_events

dormgame/presentation.py | строки 256–275

```
def accept_events(self, events: list[Any], settings: Any) -> None:
```

Получает события модели и создаёт частицы, сообщения и тряску. Например, page даёт вспышку, а died коротко трясёт камеру.

Renderer.sky

dormgame/presentation.py | строки 277–301

```
def sky(self, camera_x: float = 0, reduced: bool = False) -> None:
```

Рисует ночное небо, город, луну и дождь. Смещение дальних зданий медленнее камеры создаёт параллакс.

Renderer.window

dormgame/presentation.py | строки 303–321

```
def window(self, x: int, y: int, index: int, big: bool = False) -> None:
```

Рисует окно общежития с рамой и светом. Параметры позволяют повторять мотив без одинакового кода в нескольких местах.

Renderer.architecture

dormgame/presentation.py | строки 323–368

```
def architecture(self, level: Any) -> None:
```

Собирает фон этажа, комнаты, окна и декор из JSON. Фон визуальный и сам по себе не создаёт коллизий.

Renderer.solid

dormgame/presentation.py | строки 370–385

```
def solid(self, box: Any) -> None:
```

Рисует статическую стену или платформу по её прямоугольнику. Верхний край помогает увидеть, куда можно приземлиться.

Renderer.item

dormgame/presentation.py | строки 387–446

```
def item(self, item: Any, kind: str, active: bool = False) -> None:
```

Рисует страницу, сувенир, контрольную точку, записку или выход. Собранные предметы пропускаются; открытый выход меняет вид.

Renderer.draw_game

dormgame/presentation.py | строки 448–516

```
def draw_game(self, session: Any, settings: Any, dt: float = 0.0, debug: bool = False, fps: float = 0, steps: int = 0) -> None:
```

Плавное ведёт камеру и рисует весь игровой кадр: фон, геометрию, объекты, врагов, героя, эффекты, F3 и HUD.

Renderer.hud

dormgame/presentation.py | строки 518–554

```
def hud(self, session: Any, remaining: int, settings: Any) -> None:
```

Показывает текущий этаж, страницы, время, смерти, бонусный набор и подсказки. Читает статистику модели, не меняя правила.

Renderer.draw_debug

dormgame/presentation.py | строки 556–589

```
def draw_debug(self, level: Any, fps: float, steps: int) -> None:
```

Рисует коллайдеры, локальную навигационную сетку, маршрут, состояние врага, раскрытия A*, FPS и число физических шагов.

Renderer.backdrop

dormgame/presentation.py | строки 591–630

```
def backdrop(self, settings: Any) -> None:
```

Рисует отдельную фоновую сцену главного меню. Она не является игровым уровнем и не расходует время прохождения.

Renderer.buttons

dormgame/presentation.py | строки 632–647

```
def buttons(self, labels: list[str], selected: int, x: int, y: int, width: int, height: int = 29, gap: int = 7) -> list[pygame.Rect]:
```

Рисует список кнопок с выделением выбранной и возвращает их прямоугольники. Приложение использует их для щелчков мышью.

Renderer.front_menu

dormgame/presentation.py | строки 649–659

```
def front_menu(self, selected: int, settings: Any) -> list[pygame.Rect]:
```

Собирает название, сюжетную подпись, кнопки главного меню и byVanoGame. Возвращает области кнопок.

Renderer.shade

dormgame/presentation.py | строки 661–663

```
def shade(self, alpha: int = 190) -> None:
```

Накладывает полупрозрачное затемнение, чтобы текст поверх игры оставался читаемым.

Renderer.panel

dormgame/presentation.py | строки 665–674

```
def panel(self, title: str, subtitle: str, labels: list[str], selected: int, top: int = 85) -> list[pygame.Rect]:
```

Рисует общую панель с заголовком, подписью и кнопками. Её переиспользуют пауза и настройки.

Renderer.controls

dormgame/presentation.py | строки 676–689

```
def controls(self, selected: int) -> list[pygame.Rect]:
```

Показывает управление и цель игры. Сам метод только рисует; обработка кнопки возврата находится в приложении.

Renderer.intro

dormgame/presentation.py | строки 691–704

```
def intro(self, selected: int, settings: Any) -> list[pygame.Rect]:
```

Рисует короткое вступление и кнопку старта или пропуска. Объясняет, почему герой собирает работу и идёт на крышу.

Renderer.transition

dormgame/presentation.py | строки 706–715

```
def transition(self, session: Any, selected: int) -> list[pygame.Rect]:
```

Рисует сообщение об окончании этажа и следующем уровне. Не переключает модель самостоятельно.

Renderer.ending

dormgame/presentation.py | строки 717–743

```
def ending(self, session: Any, selected: int, settings: Any) -> list[pygame.Rect]:
```

Рисует крышу, друга с ноутбуком, статистику и результат сбора сувениров. Возвращает кнопки новой игры и меню.

Модуль app

Как окно, ввод, экраны и игровой цикл связываются вместе.

Settings.load

dormgame/app.py | строки 29–35

```
def load(cls, path: Path) -> Settings:
```

Читает сохранённые булевы настройки. При повреждённом или отсутствующем файле использует значения по умолчанию.

Settings.save

dormgame/app.py | строки 37–43

```
def save(self, path: Path) -> None:
```

Записывает настройки в папку пользователя. Если запись недоступна, игра продолжает работу с настройками в памяти.

Application.__init__

dormgame/app.py | строки 51–85

```
def __init__(self, *, smoke: bool = False, level: int = 1, scene: str = 'menu', debug: bool = False) -> None:
```

Создаёт окно Pygame, внутреннюю поверхность, представление, звук, сеанс, ввод и часы. Подготавливает начальный экран.

Application.set_scene

dormgame/app.py | строки 87–93

```
def set_scene(self, scene: str) -> None:
```

Переключает экран, выставляет паузу модели и очищает старый ввод и аккумулятор. Это общая точка перехода между состояниями приложения.

Application.start

dormgame/app.py | строки 95–99

```
def start(self, intro: bool = True) -> None:
```

Начинает новый сеанс, сбрасывает камеру и открывает вступление или игру. Используется и кнопкой повторного старта.

Application._toggle_setting

dormgame/app.py | строки 101–105

```
def _toggle_setting(self, name: str) -> None:
```

Меняет выбранную настройку на противоположную, обновляет звук и сохраняет выбор пользователя.

Application.activate

dormgame/app.py | строки 107–140

```
def activate(self, index: int) -> None:
```

Выполняет действие выбранной кнопки текущего экрана. Один индекс на разных экранах может означать начало игры, возврат или настройку.

Application.escape

dormgame/app.py | строки 142–154

```
def escape(self) -> None:
```

Определяет значение Escape для текущего экрана: пауза, продолжение, возврат или пропуск вступления.

Application._mouse_position

dormgame/app.py | строки 156–158

```
def _mouse_position(self, position: tuple[int, int]) -> tuple[float, float]:
```

Переводит координаты мыши из окна в 640×360 с учётом полос. Поэтому кнопки работают и после изменения размеров окна.

Application.handle_event

dormgame/app.py | строки 160–206

```
def handle_event(self, event: pygame.event.Event) -> None:
```

Обрабатывает события окна, клавиш и мыши. Игровые клавиши складывает в InputBuffer; потеря фокуса ставит игру на паузу.

Application.toggle_fullscreen

dormgame/app.py | строки 208–214

```
def toggle_fullscreen(self) -> None:
```

Переключает полный экран и обычное окно, запоминая прежний размер. Доступно по F11.

Application.step

dormgame/app.py | строки 216–225

```
def step(self, dt: float) -> bool:
```

Передаёт одну команду модели и после открытого выхода выбирает переход либо финал. Возвращает False, чтобы остановить догоняющие шаги при смене экрана.

Application.update

dormgame/app.py | строки 227–239

```
def update(self, dt: float) -> None:
```

Читает удержание движения, запускает FixedStepper, разбирает события и обновляет эффекты. В паузе физика не вызывается.

Application.render

dormgame/app.py | строки 241–277

```
def render(self, dt: float) -> None:
```

Выбирает рисунок текущего экрана, масштабирует внутреннюю поверхность с сохранением пропорций и показывает кадр через display.flip.

Application.run

dormgame/app.py | строки 279–294

```
def run(self, max_frames: int = 0, screenshot: Path | None = None) -> dict[str, Any]:
```

Главный цикл окна: получает события, обновляет состояние и рисует кадры до выхода. Для проверки может остановиться после заданного числа кадров.

Application.close

dormgame/app.py | строки 296–297

```
def close(self) -> None:
```

Освобождает ресурсы Pygame при завершении. Вызывается и из finally, если во время запуска произошла ошибка.

main

dormgame/app.py | строки 300–325

```
def main(argv: list[str] | None = None) -> int:
```

Разбирает параметры запуска, настраивает smoke-режим, создаёт Application и запускает цикл. finally гарантирует закрытие графической библиотеки.

Как воспроизвести материал

Скриншоты снимает tools.verify_pygame, а Word-файлы собирает tools.build_defense_docs.

Справочник ссылается на эту версию исходников. После изменения кода номера строк могут сдвинуться; для новой защиты документы нужно пересобрать.